

Cahier des charges

Projet de Porte Intelligente Connectée

1 Introduction

Ce document définit le cahier des charges du projet **Porte Intelligente** basé sur une carte **ESP32**, intégrant :

- un servomoteur pour l'ouverture et la fermeture,
- un écran LCD I2C (20x4),
- un buzzer pour les alertes sonores,
- une application mobile utilisant **Blynk 2.0** pour le contrôle à distance.

Le système permet d'ouvrir ou fermer la porte via smartphone et d'afficher l'état sur écran.

2 Objectif du projet

Le but du projet est de créer une porte motorisée contrôlée via Internet, sûre, simple à utiliser, et capable de notifier l'utilisateur de son état en temps réel.

3 Fonctionnalités principales

3.1 Fonctionnalités obligatoires

- Contrôle de la porte via l'application Blynk (ouvrir / fermer).
- Affichage de l'état de la porte sur l'écran LCD.
- Activation du buzzer lors de l'ouverture.
- Connexion WiFi automatique.
- Reconnexion automatique à Blynk.
- Sécurité de commande (interdiction d'ouverture sans action depuis Blynk).

3.2 Fonctionnalités optionnelles

- Ajout futur d'un capteur ultrason pour détecter un obstacle.
- Notification push via Blynk lors d'ouverture.

4 Composants utilisés

Composant	Rôle
ESP32 DevKit V1	Microcontrôleur principal et communication WiFi
Servomoteur (SG90 ou MG995)	Mécanisme d'ouverture/fermeture
LCD I2C 20x4	Affichage des messages du système
Buzzer	Alerte sonore lors de l'ouverture
Application Blynk 2.0	Commande mobile de la porte
Câbles et alimentation 5V	Connexions et alimentation

5 Architecture du système

5.1 Schéma logique

- L'utilisateur appuie sur un bouton dans Blynk.
- L'ESP32 reçoit la commande via le cloud Blynk.
- Le servomoteur change d'angle (0° fermé, 90° ouvert).
- Le buzzer s'active pendant 1 seconde.
- Le LCD affiche l'état actuel de la porte.

6 Exigences techniques

6.1 Réseau

- Connexion WiFi obligatoire.
- Identifiants WiFi configurables dans le code.

6.2 Logiciel

- Plateforme : Arduino IDE.
- Bibliothèques nécessaires :
 - `WiFi.h`
 - `BlynkSimpleEsp32.h`
 - `ESP32Servo.h`
 - `LiquidCrystal_I2C.h`

7 Code utilisé

Voici le code source intégré dans le projet :

```
/******  
ESP32 + Servo + LCD 20x4 + Buzzer + Blynk 2.0  
*****/  
  
// ----- CONFIG BLYNK 2.0 -----  
#define BLYNK_TEMPLATE_ID "TMPL22bXBptFc"  
#define BLYNK_TEMPLATE_NAME "aziz"  
#define BLYNK_AUTH_TOKEN "xPUCc-G60q6J4N4UWYqd2rz_RgkCjZvF"
```

```

// ----- Librairies -----
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <ESP32Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ----- CONFIG WIFI -----
char auth[] = "HD_q6hRUbTNAOP8Ic-RIyrl6yz2IBIHp"; // token
char ssid[] = "TOPNET_74B8";
char pass[] = "fkvb9j89y8";

// ----- LCD 20x4 -----
LiquidCrystal_I2C lcd(0x27, 20, 4);

// ----- SERVO -----
Servo myServo;
int servoPin = 5;
int servoAngle = 0;

// ----- BUZZER -----
int buzzerPin = 18; // Changer le pin selon votre connexion
bool doorOpen = false;

// ----- BOUTON -----
BLYNK_WRITE(V1)
{
  int state = param.asInt(); // 1 = ON , 0 = OFF

  if (state == 1) {
    // ----- OUVRIR -----
    myServo.write(0);
    doorOpen = true;

    lcd.setCursor(0, 1);
    lcd.print("Port Ouvert   "); // 5 espaces pour effacer l'ancienne ligne

    // ----- ACTIVER LE BUZZER -----
    digitalWrite(buzzerPin, HIGH);
    delay(1000); // Buzzer actif pendant 1 seconde
    digitalWrite(buzzerPin, LOW);

    Serial.println("Port Ouvert - Buzzer active");
  }
  else {
    // ----- FERMER -----
    myServo.write(90);
    doorOpen = false;

    lcd.setCursor(0, 1);

```

```

    lcd.print("Port Ferme      ");

    // ----- DESACTIVER LE BUZZER -----
    digitalWrite(buzzerPin, LOW);

    Serial.println("Port Ferme");
}
}

// =====
//                               SETUP
// =====
void setup()
{
    Serial.begin(115200);

    // ----- LCD -----
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("ESP32+SERVO+BLYNK");
    lcd.setCursor(0, 1);
    lcd.print("Port Ferme      ");

    // ----- Servo -----
    myServo.attach(servoPin);

    // ----- Buzzer -----
    pinMode(buzzerPin, OUTPUT);
    digitalWrite(buzzerPin, LOW); // S'assurer que le buzzer est éteint au démarrage

    // ----- WiFi -----
    Serial.print("Connecting to WiFi");
    WiFi.begin(ssid, pass);

    while (WiFi.status() != WL_CONNECTED) {
        Serial.print(".");
        delay(500);
    }
    Serial.println("\nWiFi Connected!");
    lcd.setCursor(0, 2);
    lcd.print("WiFi Connected      ");

    // ----- BLYNK CONFIG -----
    Blynk.config(BLYNK_AUTH_TOKEN); // Initialise Blynk sans WiFi

    Serial.print("Connecting to Blynk");

    if (!Blynk.connect(5000)) { // Timeout 5 secondes

```

```

    Serial.println("\nFailed!");
    lcd.setCursor(0, 3);
    lcd.print("Blynk Failed      ");
} else {
    Serial.println("\nConnected!");
    lcd.setCursor(0, 3);
    lcd.print("Blynk Connected    ");
}
}

// =====
//                               LOOP
// =====
void loop()
{
    // Reconnexion automatique
    if (!Blynk.connected()) {
        Blynk.connect();
    }

    Blynk.run();
}

```

8 Contraintes

- Alimentation stable 5V obligatoire.
- Le servomoteur doit être mécaniquement bien fixé.
- L'ESP32 doit être placé dans un boîtier sécurisé.

9 Critères de validation

Le projet sera considéré comme validé si :

- La porte s'ouvre et se ferme correctement via l'application.
- Le buzzer fonctionne lors de l'ouverture.
- Le LCD affiche correctement les informations.
- La connexion à Blynk est stable.

10 Conclusion

Ce cahier de charges définit toutes les étapes nécessaires pour réaliser une porte intelligente fiable, connectée et simple à utiliser. Le système pourra être amélioré grâce à des capteurs supplémentaires et des notifications avancées.